

TDD 실습 가이드

VSCode Java & C++ 개발자를 위한 단계별 TDD 실전 연습



실습 1
Arithmetic
사칙연산 TDD



실습 2
PrimeChecker
소수 판별 TDD



실습 3
Person & CalcSample
객체 TDD



실습 4
ParameterizedTest
반복 테스트



실습 5
Gilded Rose
Legacy 코드 TDD



Java: JUnit 5 · Maven · JaCoCo · Mockito



C++: GoogleTest · CMake · gcov · GMock

전체 실습 구성

실습	예제	핵심 학습 목표	Java 도구	C++ 도구
실습 1	Arithmetic — 사칙연산	TDD 기본 사이클 · 테스트 케이스 설계 · 커버리지	JUnit 5 + JaCoCo	GoogleTest + gcov
실습 2	PrimeChecker — 소수 판별	경계값 분석 · CORRECT 적용 · 단위 테스트	@ParameterizedTest	INstantiate_Test_Suite_P
실습 3	Person & CalcSample	객체 TDD · AAA 패턴 · 정적 팩토리	@BeforeEach · @DisplayName	TEST_F · SetUp()
실습 4	ParameterizedTest	@ValueSource · @EnumSource · @CsvSource	junit-jupiter-params	INstantiate_Test_Suite_P
실습 5	Gilded Rose — 레거시 TDD	Legacy 코드 이해 · 경계값 분석 · 상수 리팩토링	기존 코드 보호 테스트	기존 코드 보호 테스트

공통 TDD 진행 원칙



RED

실패하는 테스트 먼저 작성



GREEN

테스트 통과할 최소 코드



REFACTOR

중복 제거 · 가독성 개선



COVERAGE

JaCoCo / gcov 커버리지 확인



GIT

commit · push · PR · 리뷰

1

실습 #1 — Arithmetic 사칙연산 TDD

요구사항 분석 · 설계 · TDD 사이클 · 커버리지 측정

요구사항 & 클래스 설계

요구사항: 두 개의 정수를 입력받아 사칙연산(+, -, x, ÷, 나머지)을 수행하는 콘솔 프로그램을 TDD로 구현하시오.

클래스	역할	주요 메서드
ArithmeticOperations	사칙연산 로직 (Bean)	addition(x,y) / subtraction / multiply / divide / remainder
ArithmeticOperationsApp	입출력 처리 (Main)	main() — 입력받고 결과 출력
ArithmeticOperationsTest	JUnit 5 테스트	testAddition / testDivision / ...

입력/출력 예시

```

●●●
첫번째 정수값 >> 10
연산자 >> +
두번째 정수값 >> 30
=====
10 + 30 을 계산합니다.
10 + 30 = 40 입니다.
  
```

메서드 명세

메서드	매개변수	반환	설명	C++ 대응
addition(x, y)	int x, int y	x + y	덧셈	int add(int x, int y)
subtraction(x, y)	int x, int y	x - y	뺄셈	int sub(int x, int y)
multiply(x, y)	int x, int y	x * y	곱셈	int mul(int x, int y)
divide(x, y)	int x, int y != 0	x / y	나눗셈 (0 예외)	int div(int x, int y)
remainder(x, y)	int x, int y != 0	x % y	나머지	int rem(int x, int y)

테스트 케이스 설계 — 경계값 & 예외 조건

테스트 ID	테스트 목적	입력값	예상 결과	중요도
TC-ADD-001	덧셈 기본	1, 10	11	High
TC-ADD-002	덧셈 — 음수 포함	-1, -10	-11	High
TC-ADD-003	덧셈 — 0 + 0	0, 0	0	Medium
TC-SUB-001	뺄셈 기본	10, 3	7	High
TC-MUL-001	곱셈 기본	3, 4	12	High
TC-MUL-002	곱셈 — 0 포함	0, 99	0	Medium
TC-DIV-001	나눗셈 기본	10, 2	5	High
TC-DIV-002	나눗셈 — 0으로 나누기	10, 0	ArithmeticException 발생	Critical
TC-REM-001	나머지 기본	10, 3	1	High
TC-REM-002	나머지 — 음수	-10, 3	-1	Medium



CORRECT 적용: R(Range) — 음수 입력 / E(Existence) — 0으로 나누기 / C(Cardinality) — 두 정수만 입력 / O(Ordering) — 교환법칙 검증

TDD 구현 — Java (JUnit 5)

RED → GREEN → REFACTOR 사이클로 구현

```

// Java
// ● Step 1: 테스트 먼저 작성
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class ArithmeticOperationsTest {
    private ArithmeticOperations calc;

    @BeforeEach
    void setUp() {
        calc = new ArithmeticOperations();
    }

    @Test
    @DisplayName("덧셈: 1+10=11")
    void testAddition_basic() {
        assertEquals(11, calc.addition(1, 10));
    }

    @Test
    @DisplayName("덧셈: 음수+음수")
    void testAddition_negative() {
        assertEquals(-11, calc.addition(-1, -10));
    }

    @Test
    @DisplayName("나눗셈: 0으로 나누면 예외")
    void testDivision_byZero() {

```

```

// Java
// ● Step 2: 최소 구현
public class ArithmeticOperations {

    public int addition(int x, int y) {
        return x + y;
    }

    public int subtraction(int x, int y) {
        return x - y;
    }

    public int multiply(int x, int y) {
        return x * y;
    }

    public int divide(int x, int y) {
        if (y == 0) throw new
            ArithmeticException("0으로 나눌 수 없습니다");
        return x / y;
    }

    public int remainder(int x, int y) {
        if (y == 0) throw new
            ArithmeticException("0으로 나눌 수 없습니다");
        return x % y;
    }
}

```

TDD 구현 — C++ (GoogleTest)

동일한 요구사항을 C++ GoogleTest로 구현 — CMake + ctest 빌드 시스템 사용

```
// arithmetic_test.cpp
#include <gtest/gtest.h>
#include "arithmetic.hpp"

class ArithmeticTest : public ::testing::Test {
protected:
    Arithmetic calc;
};

TEST_F(ArithmeticTest, Addition_Basic) {
    EXPECT_EQ(11, calc.addition(1, 10));
}

TEST_F(ArithmeticTest, Addition_Negative) {
    EXPECT_EQ(-11, calc.addition(-1, -10));
}

TEST_F(ArithmeticTest, Division_ByZero) {
    EXPECT_THROW(calc.divide(10, 0),
        std::invalid_argument);
}
```

```
// arithmetic.hpp
#pragma once
#include <stdexcept>

class Arithmetic {
public:
    int addition(int x, int y) { return x+y; }
    int subtraction(int x, int y){ return x-y; }
    int multiply(int x, int y) { return x*y; }

    int divide(int x, int y) {
        if(y==0) throw std::invalid_argument(
            "0으로 나눌 수 없습니다");
        return x/y;
    }
    int remainder(int x, int y) {
        if(y==0) throw std::invalid_argument(
            "0으로 나눌 수 없습니다");
        return x%y;
    }
}
```

항목	Java (VSCode + Maven)	C++ (VSCode + CMake)
테스트 실행	mvn test / Testing 패널 클릭	ctest --output-on-failure / TestMate 패널
커버리지	mvn verify → JaCoCo HTML 리포트	cmake -DCMAKE_BUILD_TYPE=Coverage + lcov
Git 제출	git add . → commit → push → PR	동일

커버리지 측정 & 해석

커버리지 목표: Line Coverage 80%+ | Branch Coverage 70%+

Java — JaCoCo

```
<!-- pom.xml에 JaCoCo 플러그인 추가 -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution><goals><goal>prepare-agent</goal>
```

C++ — gcov/lcov

```
# CMakeLists.txt 에 커버리지 설정
if(CMAKE_BUILD_TYPE STREQUAL "Coverage")
  add_compile_options(--coverage -O0)
  link_libraries(--coverage)
endif()
```

실행 후 리포트 생성

커버리지 색상	의미	조치
● 초록 (Green)	테스트가 실행된 라인	이 라인은 정상 커버됨
● 갈색/빨강 (Red)	테스트가 실행되지 않은 라인	추가 테스트 케이스 필요
노란색 (Yellow)	조건부 분기 일부만 통과	true/false 양쪽 케이스 모두 필요

💡 VSCode에서 커버리지 시각화

- Java: Coverage Gutters 확장 + JaCoCo lcov.info → 에디터 라인별 색상 표시
- C++: Coverage Gutters + lcov --output-format lcov → 동일한 방식으로 확인

2

실습 #2 — PrimeChecker 소수 판별 TDD

요구사항 분석 · 설계 · TDD 사이클 · 커버리지 측정

요구사항 & 경계값 분석

요구사항: 0~1000 범위의 숫자가 소수(Prime)인지 판별하는 프로그램을 TDD로 구현하고, 경계값 분석 데이터에 맞는 테스트 케이스를 작성하시오.

CORRECT 경계 조건 분석

CORRECT	경계 조건	입력값 예시	예상 결과	중요도
C (Conformance)	정수형 입력만 허용	"abc", null	IllegalArgumentException	High
O (Ordering)	범위 내 오름차순 소수 목록	2,3,5,7,11...	순서 보장	Medium
R (Range)	0 이상 1000 이하	-1, 0, 1000, 1001	범위 외: false / 예외	High
E (Existence)	소수가 없는 범위	0, 1	소수 아님 (false)	High
C (Cardinality)	단일 입력	isPrime(2)	true	Medium
T (Time)	1000 이내 모든 소수 목록 생성	0~1000	168개 소수	Low

💡 소수 기준: 2 이상의 자연수 중 1과 자기 자신만을 약수로 갖는 수. 0과 1은 소수가 아님에 주의.

TDD 구현 — Java & C++ 병렬 비교

● RED: 테스트 먼저 → ● GREEN: 최소 구현 → ● REFACTOR: 에라토스테네스의 체 최적화

```
// PrimeCheckerTest.java - 경계값 포함
@ParameterizedTest
@ValueSource(ints = {2, 3, 5, 7, 11, 13, 997})
void isPrime_shouldReturnTrue(int n) {
    assertTrue(PrimeChecker.isPrime(n));
}

@ParameterizedTest
@ValueSource(ints = {0, 1, 4, 6, 1000, 1001})
void isPrime_shouldReturnFalse(int n) {
    assertFalse(PrimeChecker.isPrime(n));
}
```

```
// prime_checker_test.cpp
class PrimeTest : public ::testing::Test {};

// INSTANTIATE: 소수 케이스
class IsPrimeTrue : public
::testing::TestWithParam<int>{};
TEST_P(IsPrimeTrue, Check) {
    EXPECT_TRUE(isPrime(GetParam()));
}
INSTANTIATE_TEST_SUITE_P(Primes, IsPrimeTrue,
::testing::Values(2,3,5,7,11,13,997));
```

```
// PrimeChecker.java - ● 최소 구현 → ● 리팩토링 (에라토스테네스의 체 최적화)
public class PrimeChecker {
    public static boolean isPrime(int n) {
        if (n < 0) throw new IllegalArgumentException("음수 입력 불가: " + n);
        if (n < 2) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) { // O(√n) 최적화
            if (n % i == 0) return false;
        }
    }
}
```



3

실습 #3 — Person & CalcSample TDD

객체 테스트 · AAA 패턴

Person TDD — 객체 속성 테스트

요구사항: 이름(name)을 가진 Person 객체를 TDD로 구현하고, 이름 반환·변경·null 처리를 테스트하시오.



```
// PersonTest.java
class PersonTest {
    private Person person;

    @BeforeEach
    void setUp() {
        person = new Person("홍길동");
    }

    @Test
    @DisplayName("이름 반환 확인")
    void testGetName() {
        assertEquals("홍길동", person.getName());
    }

    @Test
    @DisplayName("이름 변경 후 확인")
    void testSetName() {
        person.setName("이순신");
        assertEquals("이순신", person.getName());
    }

    @Test
    @DisplayName("null 이름 생성 시 예외")
    void testNullName() {
        assertThrows(IllegalArgumentException.class,
            () -> new Person(null));
    }
}
```



```
// person_test.cpp
#include <gtest/gtest.h>
#include "person.hpp"

class PersonTest : public ::testing::Test {
protected:
    Person* person;
    void SetUp() override {
        person = new Person("홍길동");
    }
    void TearDown() override {
        delete person;
    }
};

TEST_F(PersonTest, GetName) {
    EXPECT_EQ("홍길동", person->getName());
}

TEST_F(PersonTest, SetName) {
    person->setName("이순신");
    EXPECT_EQ("이순신", person->getName());
}

TEST_F(PersonTest, NullNameThrows) {
    EXPECT_THROW(Person(nullptr),
        std::invalid_argument);
}
```

Person TDD — 객체 속성 테스트

요구사항: 이름(name)을 가진 Person 객체를 TDD로 구현하고, 이름 반환·변경·null 처리를 테스트하시오.

항목	Java (@BeforeEach)	C++ (SetUp/TearDown)
초기화	@BeforeEach void setUp()	void SetUp() override
정리	@AfterEach void tearDown()	void TearDown() override
픽스처 공유	protected 필드	protected 필드
예외 테스트	assertThrows(Type.class, lambda)	EXPECT_THROW(expr, ExceptionType)

CalcSample TDD — 정적 메서드 테스트

요구사항: CalcSample 클래스의 정적 메서드 sum(a, b)를 TDD로 구현하고 다양한 입력 조합을 테스트하시오.

```
// CalcSampleTest.java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalcSampleTest {
    @Test
    void plus() {
        // Arrange & Act & Assert
        assertEquals(3, CalcSample.sum(1, 2));
        assertEquals(5, CalcSample.sum(4, 1));
        assertEquals(0, CalcSample.sum(0, 0));
        assertEquals(-5, CalcSample.sum(-3, -2));
    }

    @ParameterizedTest
    @CsvSource({"1,2,3", "4,1,5", "0,0,0", "-3,-2,-5"})
    void sum_parameterized(int a, int b, int expected) {
        assertEquals(expected, CalcSample.sum(a, b));
    }
}
```

```
// calc_sample_test.cpp
#include <gtest/gtest.h>
#include "calc_sample.hpp"

TEST(CalcSampleTest, Plus_Basic) {
    EXPECT_EQ(3, CalcSample::sum(1, 2));
    EXPECT_EQ(5, CalcSample::sum(4, 1));
}

// 매개변수화 버전
class SumTest :
    public ::testing::TestWithParam<tuple<int,int,int>>{};
TEST_P(SumTest, Check) {
    auto [a,b,exp] = GetParam();
    EXPECT_EQ(exp, CalcSample::sum(a,b));
}
INSTANTIATE_TEST_SUITE_P(Cases, SumTest,
    ::testing::Values(
        make_tuple(1,2,3), make_tuple(4,1,5)));
```

CalcSample TDD — 정적 메서드 테스트

요구사항: CalcSample 클래스의 정적 메서드 sum(a, b)를 TDD로 구현하고 다양한 입력 조합을 테스트하시오.

```
// CalcSample.java - ● GREEN
public class CalcSample {
    public static int sum(int a, int b) { return a +
b; }
}
```

```
// calc_sample.hpp - ● GREEN
struct CalcSample {
    static int sum(int a, int b) { return a + b; }
};
```


4

실습 #4 —

ParameterizedTest / ApprovalTests 심화

@ValueSource · @EnumSource · @CsvSource · @MethodSource

프로젝트 구조 & pom.xml 설정

- DayOfWeek.java — 요일 Enum (MONDAY ~ SUNDAY)
- WeekendChecker.java — 주어진 요일이 주말 (SATURDAY, SUNDAY)인지 판별
- FruitRanker.java — 과일과 등급 검증 (null·빈 값·양 수 체크)
- ParameterizedTestTest.java — 다양한 JUnit 5 매개 변수화 테스트

```

<!-- pom.xml - ParameterizedTest 의존성 -->
<dependencies>
  <!-- JUnit 5 기본 -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
  <!-- 매개변수화 테스트 모듈 ★ -->

```

어노테이션	사용 목적	예시	C++ 대응
@ValueSource	단일 값 목록	@ValueSource(ints={1,2,3})	Values(1,2,3)
@EnumSource	Enum 전체 또는 선택	@EnumSource(DayOfWeek.class)	Values(MON,TUE,...)
@CsvSource	입력/기댓값 쌍	@CsvSource({"a,1","b,2"})	make_tuple("a",1)
@MethodSource	복잡한 객체 스트림	@MethodSource("provideArgs")	별도 제공 함수
@NullSource	null 입력 테스트	@NullSource	—

⚠ junit-jupiter-params 가 pom.xml에 없으면 @ParameterizedTest 사용 시 컴파일 에러 발생. 반드시 <scope>test</scope> 로 추가하세요.

ParameterizedTest 전체 예제

```

// ParameterizedTestTest.java

// 1. 문자열 목록 - @ValueSource
@ParameterizedTest
@ValueSource(strings = {"hong", "kim", "park"})
void printName(String name) {
    assertNotNull(name);
    assertFalse(name.isBlank());
}

// 2. Enum 전체 - @EnumSource
@ParameterizedTest
@EnumSource(DayOfWeek.class)
void printDayOfWeek(DayOfWeek day) {
    assertNotNull(day);
}

// 3. Enum 선택 - names 필터
@ParameterizedTest
@EnumSource(value=DayOfWeek.class,
    names={"MONDAY", "WEDNESDAY", "FRIDAY"})
void printWeekdays(DayOfWeek day) {
    assertFalse(
        day==DayOfWeek.SATURDAY || day==DayOfWeek.SUNDAY);
}

// 4. CSV 다중 입력 - @CsvSource
@ParameterizedTest
@CsvSource({
    "apple, 1",
    "banana, 2",
    "lemon, lime, 3"
})
void testWithCsvSource(String fruit, int rank) {
    assertNotNull(fruit);
    assertTrue(rank > 0);
}

// 5. 주말 판별 - 성공/실패 혼합
@ParameterizedTest(name="{0} 주말여부")
@EnumSource(value=DayOfWeek.class,
    names={"SATURDAY", "MONDAY", "SUNDAY"})
void testIsWeekend(DayOfWeek day) {
    // MONDAY는 주말 아님 → 이 테스트는 실패!
    // → 의도적 실패로 TDD 진행
    assertTrue(
        day==DayOfWeek.SATURDAY || day==DayOfWeek.SUNDAY,
        day+"는 주말이 아닙니다.");
}

// ⚠ MONDAY가 포함돼 테스트 실패 → 의도된 것
// → WeekendChecker 로직 점검 기회

```



테스트 실패(MONDAY 케이스)는 의도된 것 — TDD에서 실패 케이스로 WeekendChecker 로직의 경계값을 확인하는 학습 포인트

ParameterizedTest 전체 예제

#	Java 어노테이션	C++ 대응	내용
1	@ValueSource	Values(...)	문자열 목록
2	@EnumSource (전체)	Values(모든 enum)	Enum 전체 순회
3	@EnumSource (names 필터)	Values(선택 enum)	평일만 필터
4	@CsvSource	make_tuple(...)	입력/기댓값 쌍
5	성공/실패 혼합	동일	MONDAY 포함 → 의도적 실패 (TDD)

ApprovalTests — 복잡한 출력 결과 스냅샷 검증

1

테스트 실행 → actual을 .received.txt 저장

2

.approved.txt 와 자동 비교

3

일치 → 통과 / 불일치 → FAIL + diff 뷰어

C++ (Google Test / GMock)

```
// — CMake - ApprovalTests 설치 —————
# FetchContent로 헤더 전용 라이브러리 추가
FetchContent_Declare(
    ApprovalTests
    GIT_REPOSITORY
        https://github.com/approvals/
        ApprovalTests.cpp.git
    GIT_TAG v.10.13.0
)
FetchContent_MakeAvailable(ApprovalTests)

target_link_libraries(all_tests
    ApprovalTests::ApprovalTests)

// — 기본 사용법 —————
#define APPROVALS_GOOGLETEST
#include "ApprovalTests.hpp"
```

Java 대응 (JUnit 5 / Mockito)

```
// Java Approval - Maven 의존성
// <dependency>
//   <groupId>com.approvaltests</groupId>
//   <artifactId>approvaltests</artifactId>
//   <version>9.5.0</version>
//   <scope>test</scope>
// </dependency>

import org.approvaltests.Approvals;

// 기본 사용법
@Test
public void testSortWords() {
    String input =
        "banana apple Orange Grape";
    String result =
        StringProcessor.process(input);

    Approvals.verify(result);
    // → ClassName.methodName
    //   .approved.txt 와 비교
}
```

approved.txt = 기준(Golden Master) 파일을 직접 생성하거나, received.txt 내용을 확인 후 rename하여 승인 | diff 뷰어: AraxisMerge / Beyond Compare / VSCode

5

실습 #5 — Gilded Rose Legacy TDD

레거시 코드 이해 · 경계값 분석 · 상수 리팩토링 · 승인 테스트

C++17

Google Test

ApprovalTests.cpp

CMake

Java 참고: Java 17+

Java 참고: JUnit 5

Java 참고: ApprovalTests Java

STEP 1
요구사항 분석

STEP 2
레거시 코드 이해

STEP 3
경계값 테스트

STEP 4
상수 리팩토링

STEP 5
승인 테스트

STEP 1 — Gilded Rose 요구사항 & 비즈니스 규칙

레거시 코드를 수정하기 전, 반드시 요구사항을 완전히 이해해야 한다

📖 배경 스토리

여관(Gilded Rose)을 운영하는 Allison은 상품들의 품질(Quality)을 매일 갱신하는 시스템을 운용 중이다. 레거시 코드를 건드리지 않고 "Conjured" 아이템 타입 추가 기능을 구현해야 한다.

📦 일반 아이템

- 매일 sellIn(판매기한)이 1씩 감소
- 매일 quality(품질)가 1씩 감소
- sellIn이 0 이하면 quality 2배 감소

🎫 Backstage Passes

- 10일 이상 남음: +1/일
- 10일~6일: +2/일, 5일~1일: +3/일
- 콘서트 후(sellIn=0): quality = 0

🧀 Aged Brie

- 오래될수록 quality 증가 (+1/일)
- sellIn 지나면 quality 2배 증가
- quality 최댓값: 50

✨ Conjured (신규)

- 매일 quality 2배 감소 (일반의 2x)
- sellIn 지나면 4배 감소
- 추가 구현 대상 — TDD로 작성!

🔨 Sulfuras

- 전설 아이템: sellIn-quality 변화 없음
- quality = 80 고정 (항상)
- 절대 판매되지 않음

핵심 제약: quality는 0 미만이 될 수 없고 50을 초과할 수 없다 (Sulfuras 제외) ← 테스트 경계값의 핵심!

STEP 2 — 레거시 코드 이해 & 문제점 파악

테스트 없이 수정하면 안 된다 — 먼저 레거시를 이해하고 테스트로 보호막을 만들어라

C++ (Google Test)

```
// GildedRose.h — 레거시 코드 (수정 금지 구역!)
#pragma once
#include <string>
#include <vector>

struct Item {
    std::string name;
    int sellIn;    // 판매까지 남은 일수
    int quality;   // 아이템 품질 (0~50)
    Item(std::string name, int sellIn, int quality)
        : name(std::move(name)), sellIn(sellIn), quality(quality) {}
};
```

⚠ 레거시 코드 문제점

- 중첩 if문 7단계 → 가독성 최악
- 매직 스트링 ("Aged Brie" 등) 하드코딩
- 단일 updateQuality() 메서드에 모든 로직
- 테스트 코드 전무 → 안전 수정 불가
- Conjured 추가 시 어디를 건드려야 할지 모름

레거시 코드 접근 원칙: 1) 이해 → 2) 테스트로 보호 → 3) 리팩토링 → 4) 기능 추가 (역순으로 하면 반드시 버그 발생!)

STEP 2 — 레거시 코드 이해 & 문제점 파악

C++ (Google Test)

```
class GildedRose {
public:
    std::vector<Item>& items;
    explicit GildedRose(std::vector<Item>& items) : items(items) {}

    void updateQuality() {
        for (auto& item : items) {
            if (item.name != "Aged Brie" &&
                item.name != "Backstage passes to a TAFKAL80ETC concert") {
                if (item.quality > 0) {
                    if (item.name != "Sulfuras, Hand of Ragnaros")
                        item.quality--;
                }
            } else {
                if (item.quality < 50) {
                    item.quality++;
                    if (item.name == "Backstage passes...") {
                        if (item.sellIn < 11) {
                            if (item.quality < 50) item.quality++;
                        }
                        if (item.sellIn < 6) {
                            if (item.quality < 50) item.quality++;
                        }
                    }
                }
            }
        }
    }
}
```

Java 대응 (JUnit 5)

```
// Java 레거시 (동일 구조)
public class GildedRose {
    Item[] items;
    public GildedRose(Item[] items){
        this.items = items;
    }
    public void updateQuality(){
        for (Item item : items) {
            if (!item.name.equals(
                "Aged Brie") && !item.name
                .equals("Backstage passes")){
                if (item.quality > 0){
                    if (!item.name.equals(
                        "Sulfuras"))
                        item.quality--;
                }
            } else {
                if (item.quality < 50)
                    item.quality++;
                // ... 더 중첩된 if들 ...
            }
            if (!item.name.equals("Sulfuras"))
                item.sellIn--;
            // ... 판매기한 이후 처리 ...
        }
    }
}
```

STEP 2 — 레거시 코드 이해 & 문제점 파악

C++ (Google Test)

```
if (item.name != "Sulfuras, Hand of Ragnaros") item.sellIn--;  
if (item.sellIn < 0) {  
    if (item.name != "Aged Brie") {  
        if (item.name != "Backstage passes...") {  
            if (item.quality > 0) {  
                if (item.name != "Sulfuras, Hand of Ragnaros")  
                    item.quality--;  
            }  
        } else {  
            item.quality = item.quality - item.quality; // 0  
        }  
    } else {  
        if (item.quality < 50) item.quality++;  
    }  
}  
}  
};
```

STEP 3 — 경계값 분석 & 테스트 케이스 설계

CORRECT 원칙 — Range·Boundary·Error 조건을 먼저 테스트로 정의한다

경계값 분석표 (Right-BICEP + CORRECT)

아이템	경계 조건	예상 결과	테스트 우선순위
일반	quality=0일 때 → 더 이상 감소 안 함 sellIn=0 → quality 2배 감소 quality=50 → 정상 감소	quality >= 0 보장 sellIn < 0: quality -= 2 quality = 49	★★★
Aged Brie	quality=50 → 더 이상 증가 안 함 sellIn=0 → quality +2	quality <= 50 보장 sellIn < 0: quality += 2	★★★
Backstage Passes	sellIn=11 → +1 sellIn=10 → +2 sellIn=5 → +3 sellIn=0 → quality=0	각 구간 경계에서 +1/+2/+3 concert 후 = 0	★★★
Sulfuras	sellIn, quality 항상 불변 quality = 80 고정	업데이트 후에도 동일	★★☆
Conjured 🍄	quality=1 → 2씩 감소 시 0이 됨 quality=0 → 더 이상 감소 안 함 sellIn=0 → 4배 감소	quality >= 0 보장 0에서 멈춤 quality -= 4	★★★ (미구현!)

STEP 3 — 경계값 분석 & 테스트 케이스 설계

CORRECT 원칙 — Range-Boundary-Error 조건을 먼저 테스트로 정의한다



경계값 테스트 코드 — C++ 위주

C++ (Google Test)

```
// 경계값: quality는 0 미만이 되어선 안 된다
TEST(GildedRoseTest, QualityNeverNegative) {
    std::vector<Item> items = {Item("Normal Item", 5, 0)};
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_GE(items[0].quality, 0); // quality >= 0 보장
}

// 경계값: quality는 50을 초과해선 안 된다
TEST(GildedRoseTest, AgedBrieQualityMax50) {
    std::vector<Item> items = {Item("Aged Brie", 5, 50)};
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_LE(items[0].quality, 50); // quality <= 50 보장
}

// 경계값: sellIn 지나면 quality 2배 감소
TEST(GildedRoseTest, NormalItemDegradesTwiceAfterSellDate) {
    std::vector<Item> items = {Item("Normal Item", 0, 10)};
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_EQ(8, items[0].quality); // 10 - 2 = 8
}
```

Java 대응 (JUnit 5)

```
// Java 경계값 테스트
@Test void qualityNeverNegative() {
    Item[] items = {
        new Item("Normal",5,0)};
    GildedRose gr = new GildedRose(items);
    gr.updateQuality();
    assertTrue(items[0].quality >= 0);
}

@Test void agedBrieMax50() {
    Item[] items = {
        new Item("Aged Brie",5,50)};
    new GildedRose(items).updateQuality();
    assertTrue(items[0].quality <= 50);
}

@Test void normalDegradesTwice() {
    Item[] items = {
        new Item("Normal",0,10)};
    new GildedRose(items).updateQuality();
    assertEquals(8, items[0].quality);
}
```

STEP 3 계속 — Backstage Passes 경계값 & 파라미터화 테스트

sellIn 구간별 quality 증가량 — 경계(10 일, 5 일, 0 일)가 핵심

sellIn > 10

+1

$6 \leq \text{sellIn} \leq 10$

+2

$1 \leq \text{sellIn} \leq 5$

+3

sellIn = 0

= 0

← 시간 경과 (sellIn 감소)

C++ (Google Test)

```
// Backstage Passes 경계값 — 파라미터화 테스트 (C++)
class BackstagePassTest
: public ::testing::TestWithParam<
    std::tuple<int, int, int>> {} {
// <sellIn, initialQuality, expectedQuality>

TEST_P(BackstagePassTest, QualityUpdate) {
    auto [sellIn, initQ, expectedQ] = GetParam();
    std::vector<Item> items = {
        Item("Backstage passes to a TAFKAL80ETC concert",
            sellIn, initQ)
    };
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_EQ(expectedQ, items[0].quality);
}
```

Java 대응 (JUnit 5)

```
// Java — @ParameterizedTest 대응
@ParameterizedTest(
    name="sellIn={0},q={1}→{2}")
@CsvSource({
    "15, 20, 21", // > 10: +1
    "11, 20, 21", // 경계
    "10, 20, 22", // +2 시작
    " 6, 20, 22", // 경계
    " 5, 20, 23", // +3 시작
    " 1, 20, 23", // 경계
    " 0, 20,  0", // concert 후
    " 5, 50, 50", // 상한 50
    " 0, 50,  0" // 0으로
})
```

STEP 3 계속 — Backstage Passes 경계값 & 파라미터화 테스트

← 시간 경과 (sellIn 감소)

C++ (Google Test)

```
INSTANTIATE_TEST_SUITE_P(
    BackstageBoundary, BackstagePassTest,
    ::testing::Values(
        // (sellIn, initQ, expectedQ)
        std::make_tuple(15, 20, 21), // > 10: +1
        std::make_tuple(11, 20, 21), // 경계: sellIn=11 → +1
        std::make_tuple(10, 20, 22), // 경계: sellIn=10 → +2
        std::make_tuple( 6, 20, 22), // 경계: sellIn=6 → +2
        std::make_tuple( 5, 20, 23), // 경계: sellIn=5 → +3
        std::make_tuple( 1, 20, 23), // 경계: sellIn=1 → +3
        std::make_tuple( 0, 20,  0), // 경계: sellIn=0 → quality=0
        std::make_tuple( 5, 50, 50), // quality 상한 50
        std::make_tuple( 0, 50,  0), // concert 후 → 0
    )
);
```

Java 대응 (JUnit 5)

```
void backstageTest(
    int sellIn, int initQ, int exp) {
    Item[] items = { new Item(
        "Backstage passes...",
        sellIn, initQ) };
    new GildedRose(items).updateQuality();
    assertEquals(exp, items[0].quality);
}
```

경계값 포인트: sellIn=11/10 (1→2 전환), sellIn=6/5 (2→3 전환), sellIn=1/0 (3→0 전환) ← Off-by-one 버그가 가장 자주 발생!

STEP 4 — 상수 리팩토링 (매직 스트링 → 명명 상수)

하드코딩된 문자열을 `const/constexpr`로 추출 → 오타 방지 + 컴파일 타임 검증

✗ Before — 매직 스트링 하드코딩 (위험!)

C++ (Google Test)

```
// ✗ Before - 매직 스트링이 코드 전체에 흩어져 있음
void GildedRose::updateQuality() {
    for (auto& item : items) {
        if (item.name != "Aged Brie" && // 오타 위험!
            item.name != "Backstage passes to a TAFKAL80ETC concert") {
            if (item.name != "Sulfuras, Hand of Ragnaros") {
                item.quality--;
            }
        }
        // "Aged Brie"가 여러 군데 반복 → 하나라도 오타나면 버그!
    }
}
```

STEP 4 — 상수 리팩토링 (매직 스트링 → 명명 상수)

✓ After — constexpr 명명 상수로 추출 (안전!)

C++ (Google Test)

```
// ✓ After — 상수를 별도 헤더에 정의 (GildedRoseConstants.h)
namespace GildedRoseConstants {
    inline constexpr const char* AGED_BRIE = "Aged Brie";
    inline constexpr const char* BACKSTAGE_PASS = "Backstage passes to a TAFKAL80ETC concert";
    inline constexpr const char* SULFURAS = "Sulfuras, Hand of Ragnaros";
    inline constexpr const char* CONJURED = "Conjured Mana Cake";
    inline constexpr int MAX_QUALITY = 50;
    inline constexpr int MIN_QUALITY = 0;
    inline constexpr int SULFURAS_QUALITY = 80;
}

// ✓ 리팩토링된 updateQuality()
void GildedRose::updateQuality() {
    using namespace GildedRoseConstants;
    for (auto& item : items) {
        if (item.name != AGED_BRIE && item.name != BACKSTAGE_PASS) {
            if (item.name != SULFURAS && item.quality > MIN_QUALITY)
                item.quality--;
        }
        // ... 나머지 로직도 상수 사용
    }
}

// ✓ 상수 리팩토링 후 테스트 (기존 테스트 전부 PASS 확인!)
TEST(ConstantsTest, MaxQualityDefined) {
    EXPECT_EQ(50, GildedRoseConstants::MAX_QUALITY);
    EXPECT_EQ(0, GildedRoseConstants::MIN_QUALITY);
}
```

Java 대응 (JUnit 5)

```
// Java — static final 상수로 추출
public class GildedRose {
    // 상수 정의
    static final String AGED_BRIE = "Aged Brie";
    static final String BACKSTAGE = "Backstage passes to a " + "TAFKAL80ETC concert";
    static final String SULFURAS = "Sulfuras, Hand of Ragnaros";
    static final String CONJURED = "Conjured Mana Cake";
    static final int MAX_QUALITY = 50;
    static final int MIN_QUALITY = 0;

    // 리팩토링된 메서드
    public void updateQuality() {
        for (Item item : items) {
            if (!item.name.equals(AGED_BRIE) && !item.name.equals(BACKSTAGE)) {
                if (!item.name.equals(SULFURAS) && item.quality > MIN_QUALITY)
                    item.quality--;
            }
        }
    }
}
```

C++ constexpr = 컴파일 타임 상수 (Java의 static final과 동일한 역할) | inline constexpr = 헤더 파일에서 ODR 위반 없이 정의 가능

STEP 5 — 승인 테스트 (ApprovalTests) — 레거시 코드 보호막 구축

모든 아이템 × 30일 시뮬레이션 결과를 Golden Master 파일로 고정한다

1 테스트 첫 실행
→ .received.txt 생성

2 결과 검토 후
.received → .approved 복사 (승인)

3 이후 실행마다
.approved.txt와 자동 비교

4 불일치 시 FAIL
+ diff 뷰어로 변경점 확인

C++ (Google Test)

```
// CMakeLists.txt - ApprovalTests.cpp 추가
FetchContent_Declare(ApprovalTests
  GIT_REPOSITORY https://github.com/approvals/ApprovalTests.cpp.git
  GIT_TAG v.10.13.0)
FetchContent_MakeAvailable(ApprovalTests)

target_link_libraries(gilded_rose_tests
  GTest::gtest_main ApprovalTests::ApprovalTests)

// GildedRoseApprovalTest.cpp - 30일 시뮬레이션 승인 테스트
#define APPROVALS_GOOGLETEST
#include "ApprovalTests.hpp"
#include "GildedRose.h"
#include <sstream>
```

Java 대응 (JUnit 5)

```
// Java - ApprovalTests 설정 (Maven)
// <dependency>
//   <groupId>com.approvaltests</groupId>
//   <artifactId>approvaltests</artifactId>
//   <version>9.5.0</version>
// </dependency>
```

무조건 실패(always-failing test): 처음엔 approved.txt가 없어 FAIL → received.txt 검토 후 rename → 이후 레거시 코드 보호막 완성!

STEP 5 — 승인 테스트 (ApprovalTests) — 레거시 코드 보호막 구축

C++ (Google Test)

```
// 30일 시뮬레이션 출력 생성 함수
std::string simulate30Days(std::vector<Item> items) {
    GildedRose gr(items);
    std::ostringstream oss;
    for (int day = 0; day <= 30; ++day) {
        oss << "----- day " << day << " -----
";
        for (const auto& item : gr.items)
            oss << item.name << ", " << item.sellIn
                << ", " << item.quality << "
";
        if (day < 30) gr.updateQuality();
    }
    return oss.str();
}

TEST(GildedRoseApproval, ThirtyDaySimulation) {
    std::vector<Item> items = {
        Item("+5 Dexterity Vest",    10, 20),
        Item("Aged Brie",            2,  0),
        Item("Elixir of the Mongoose", 5,  7),
        Item("Sulfuras, Hand of Ragnaros", 0, 80),
        Item("Sulfuras, Hand of Ragnaros", -1, 80),
        Item("Backstage passes to a TAFKAL80ETC concert", 15, 20),
        Item("Backstage passes to a TAFKAL80ETC concert", 10, 49),
        Item("Backstage passes to a TAFKAL80ETC concert", 5, 49),
        Item("Conjured Mana Cake",    3,  6), // 신규!
    };
    // approved.txt와 자동 비교 - 한 줄로 전체 동작 고정!
    ApprovalTests::Approvals::verify(simulate30Days(items));
}
```

Java 대응 (JUnit 5)

```
// 30일 시뮬레이션
public String simulate30Days(
    Item[] startItems) {
    GildedRose gr = new GildedRose(startItems);
    StringBuilder sb = new StringBuilder();
    for (int d = 0; d <= 30; d++) {
        sb.append("--- day ").append(d)
          .append(" ---
");
        for (Item i : gr.items)
            sb.append(i.name).append(", ")
              .append(i.sellIn).append(", ")
              .append(i.quality).append("
");
        if (d < 30) gr.updateQuality();
    }
    return sb.toString();
}

// 승인 테스트 (한 줄!)
@Test
public void thirtyDaySimulation() {
    Item[] items = {
        new Item("+5 Dex Vest", 10, 20),
        new Item("Aged Brie", 2, 0),
        new Item("Sulfuras", 0, 80),
        // ...
    };
    Approvals.verify(
        simulate30Days(items));
    // ← .approved.txt와 자동 비교
}
```

STEP 5 계속 — Conjured 기능 TDD로 추가 (Red→Green→Refactor)

승인 테스트로 레거시를 보호한 뒤, Conjured를 안전하게 TDD로 구현한다



C++ (Google Test)

```
// ① RED - 실패하는 테스트 먼저 작성
TEST(ConjuredTest, DegradesTwiceNormal) {
    std::vector<Item> items = {
        Item("Conjured Mana Cake", 10, 20)
    };
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_EQ(18, items[0].quality); // 20 - 2 = 18 ← 아직 실패!
}

TEST(ConjuredTest, DegradesTwiceAfterSellDate) {
    std::vector<Item> items = {
        Item("Conjured Mana Cake", 0, 10)
    };
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_EQ(6, items[0].quality); // 10 - 4 = 6 ← 아직 실패!
}

TEST(ConjuredTest, QualityNeverNegative) {
    std::vector<Item> items = {
        Item("Conjured Mana Cake", 5, 1)
    };
    GildedRose gr(items);
    gr.updateQuality();
    EXPECT_EQ(0, items[0].quality); // 1 - 2 → 0 (음수 방지)
}
```

Java 대응 (JUnit 5)

```
// ① Red - 실패 테스트
@Test void conjuredDegradesTwice() {
    Item[] items = {
        new Item("Conjured Mana Cake",
            10, 20));
    new GildedRose(items).updateQuality();
    assertEquals(18, items[0].quality);
}

@Test void conjuredAfterSellDate() {
    Item[] items = {
        new Item("Conjured Mana Cake",
            0, 10));
    new GildedRose(items).updateQuality();
    assertEquals(6, items[0].quality);
}
```

STEP 5 계속 — Conjured 기능 TDD로 추가 (Red→Green→Refactor)

C++ (Google Test)

```
// ㉔ GREEN - Conjured 처리 추가 (최소한의 코드)
void GildedRose::updateQuality() {
    using namespace GildedRoseConstants;
    for (auto& item : items) {
        if (item.name == CONJURED) {
            // 2배 감소, sellIn 지나면 4배
            int degradeBy = (item.sellIn > 0) ? 2 : 4;
            item.quality = std::max(MIN_QUALITY,
                                    item.quality - degradeBy);

            item.sellIn--;
            continue;
        }
        // ... 기존 로직 유지 ...
    }
}

// ㉕ REFACTOR - 공통 로직 추출
auto degrade = [&](Item& item, int amount) {
    item.quality = std::max(MIN_QUALITY, item.quality - amount);
};
auto upgrade = [&](Item& item, int amount) {
    item.quality = std::min(MAX_QUALITY, item.quality + amount);
};
```

Java 대응 (JUnit 5)

```
// ㉔ Green - 최소 구현
if (item.name.equals(CONJURED)) {
    int deg = (item.sellIn > 0) ? 2 : 4;
    item.quality =
        Math.max(MIN_QUALITY,
                 item.quality - deg);
    item.sellIn--;
    continue;
}

// ㉕ Refactor - 헬퍼 메서드
private void degrade(Item item, int n){
    item.quality =
        Math.max(MIN_QUALITY,
                 item.quality - n);
}
private void upgrade(Item item, int n){
    item.quality =
        Math.min(MAX_QUALITY,
                 item.quality + n);
}
```

Conjured 추가 후 승인 테스트 재실행 → 새 .received.txt 확인 → 맞으면 .approved.txt 업데이트 → 레거시 보호막 갱신 완료!

VSCode 실습 환경 설정 & 실행 가이드

CMake + Google Test + ApprovalTests.cpp 통합 설정

C++ (Google Test)

```
# 프로젝트 구조
gilded_rose/
├── CMakeLists.txt
├── include/
│   ├── GildedRose.h
│   └── GildedRoseConstants.h
├── src/
│   └── GildedRose.cpp
└── tests/
    ├── GildedRoseTest.cpp      # 단위 테스트 (경계값)
    └── GildedRoseApprovalTest.cpp # 승인 테스트 (30일)

# CMakeLists.txt (핵심 부분)
cmake_minimum_required(VERSION 3.14)
project(GildedRose CXX)
set(CMAKE_CXX_STANDARD 17)
```

VSCode 필수 확장

ms-vscode.cpptools-extension-pack

ms-vscode.cmake-tools

matepek.vscode-catch2-test-adapter

→ Test Explorer에서 ▶ 클릭 실행

Ctrl+Shift+P → 'CMake: Run Tests'

Java 대응 (JUnit 5)

```
// Java 프로젝트 구조 (Maven)
gilded_rose_java/
├── pom.xml
├── src/
│   ├── main/java/
│   │   ├── GildedRose.java
│   │   └── Item.java
│   └── test/java/
│       ├── GildedRoseTest.java
│       └── GildedRoseApprovalTest.java
```

VSCode 실습 환경 설정 & 실행 가이드

C++ (Google Test)

```
include(FetchContent)
# Google Test
FetchContent_Declare(googletest
  URL https://github.com/google/googletest/archive/v1.14.0.zip)
# ApprovalTests.cpp
FetchContent_Declare(ApprovalTests
  GIT_REPOSITORY
    https://github.com/approvals/ApprovalTests.cpp.git
  GIT_TAG v.10.13.0)
FetchContent_MakeAvailable(googletest ApprovalTests)

add_library(gilded_rose src/GildedRose.cpp)
target_include_directories(gilded_rose PUBLIC include)

add_executable(gilded_rose_tests
  tests/GildedRoseTest.cpp
  tests/GildedRoseApprovalTest.cpp)
target_link_libraries(gilded_rose_tests
  gilded_rose
  GTest::gtest_main
  ApprovalTests::ApprovalTests)

include(GoogleTest)
gtest_discover_tests(gilded_rose_tests)

# 빌드 & 실행
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j4
ctest --test-dir build -V
```

Java 대응 (JUnit 5)

```
// pom.xml 의존성
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.approvaltests</groupId>
    <artifactId>approvaltests</artifactId>
    <version>9.5.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>

// VSCode Java 확장:
// - Extension Pack for Java
// - Test Runner for Java
```

실습 #5 종합 정리 — Gilded Rose Legacy TDD 핵심 패턴

STEP 1



요구사항 분석

5가지 아이템 규칙 + Conjured
추가 목표 파악
→ 비즈니스 규칙을 먼저 문서화

STEP 2



레거시 이해

중첩 if, 매직 스트링, 테스트 부재 파악
→ 절대 바로 수정하지 말 것!

STEP 3



경계값 테스트

quality 0/50, sellIn 10/5/0 경계 테스트
→ TEST_P 파라미터화로 빈틈 없이

STEP 4



상수 리팩토링

매직 스트링 → constexpr 명명 상수
→ 기존 테스트 전부 PASS 확인 필수

STEP 5



승인 테스트 + TDD

30일 시뮬레이션 Golden Master 고정
→ Conjured를 Red-Green-Refactor로 추가

C++ (Google Test) vs Java (JUnit 5) — 핵심 비교

항목	C++ (Google Test)	Java (JUnit 5)
레거시 코드	GildedRose.h/.cpp — 헤더/구현 분리	GildedRose.java — 단일 파일
상수 정의	inline constexpr const char* / int	static final String / int
파라미터화	TEST_P + INSTANTIATE_TEST_SUITE_P	@ParameterizedTest + @CsvSource
승인 테스트	ApprovalTests.cpp — Approvals::verify()	approvaltests — Approvals.verify()
빌드/실행	CMake + ctest --test-dir build -V	mvn test / VSCode Test Runner

레거시 TDD 황금 규칙: 이해 → 테스트 보호막 → 상수 리팩토링 → 승인 테스트 → 기능 추가 (이 순서를 절대 바꾸지 말 것!)

6

Git 협업 워크플로우

실습별 브랜치 전략 · PR · 코드 리뷰

실습별 Git 전략 & 커밋 컨벤션

실습	Repository 명	브랜치 전략	PR 목적
실습 1 (Arithmetic)	Arithmetic_XX	main 브랜치에 직접 push	기본 TDD 사이클 연습
실습 2 (PrimeChecker)	PrimeChecker_XX	main 브랜치에 직접 push	경계값 분석 + 커버리지
실습 3 (Person)	Person_XX / CalcSample_XX	main 브랜치에 직접 push	객체 TDD + AAA 패턴
실습 4 (Parameterized)	ParameterizedTest_XX	dev → PR → main	ParameterizedTest + 리뷰
실습 5 (Gilded Rose)	GildedRose_XX	dev → PR → main	Legacy TDD + 코드 리뷰



```
# 실습 4/5 브랜치 전략 (PR + 리뷰)
git checkout -b dev
# ... 테스트 코드 작성 ...
git add -A
git commit -m "test: Add ParameterizedTest for DayOfWeek"
git push origin dev

# GitHub에서 PR: dev → main 생성
# → 리뷰어가 코드 리뷰 후 머지
git checkout main
git pull origin main
```

커밋 메시지 컨벤션

타입	설명	예시
test	테스트 코드 추가	test: Add ArithmeticTest
feat	기능 구현	feat: Implement isPrime()
refactor	리팩토링	refactor: Extract constant
fix	버그 수정	fix: Fix quality overflow
docs	문서 작성	docs: Update README TODO

전체 실습 비교 정리

실습	핵심 TDD 학습 요소	Java 주요 API	C++ 주요 API
1 Arithmetic	RED→GREEN→REFACTOR 기본 사이클 테스트 케이스 설계 · JaCoCo 커버리지	@BeforeEach · assertThrows jacoco-maven-plugin	TEST_F · EXPECT_THROW gcov + lcov
2 PrimeChecker	경계값 분석 · CORRECT 적용 @ParameterizedTest + @ValueSource	@ParameterizedTest @ValueSource(ints={...})	INstantiateTestSuite_P ::testing::Values(...)
3 Person / CalcSample	객체 TDD · 픽스처 분리 @DisplayName · static 메서드 테스트	@BeforeEach · @DisplayName @CsvSource	SetUp() · TearDown() TestWithParam<tuple>
4 ParameterizedTest	반복 테스트 심화 @EnumSource · @CsvSource · @MethodSource	junit-jupiter-params 의존성 @EnumSource · @CsvSource	INstantiateTestSuite_P ::testing::Combine(...)
5 Gilded Rose	Legacy 코드 이해 · 승인 테스트 상수 리팩토링 · 버그 발견	static final 상수 추출 ApprovalTest (ApprovalTests 라이브러리)	const 멤버 변수 분리 Golden Master 테스트

TDD 3법칙 — ① 실패하는 테스트 먼저 ② 컴파일 실패하지 않을 정도로만 테스트 ③ 실패 테스트를 통과할 정도로만 구현
(Robert C. Martin)

수고하셨습니다! 🎉

"Test the program before you write it." — Kent Beck

- ✅ 실습 1: Arithmetic — 사칙연산 TDD · 커버리지 측정 · 0 나눗셈 예외
- ✅ 실습 2: PrimeChecker — 경계값 분석 · CORRECT · @ParameterizedTest
- ✅ 실습 3: Person / CalcSample — 객체 TDD · AAA · @BeforeEach
- ✅ 실습 4: ParameterizedTest — @ValueSource · @EnumSource · @CsvSource
- ✅ 실습 5: Gilded Rose — Legacy TDD · 상수 리팩토링 · 버그 발견



Java

JUnit 5 · Mockito · JaCoCo · Maven



C++

GoogleTest · GMock · gcov · CMake